



Yes, walk through a vector add example and show the tuning steps

Here is a step-by-step, **hands-on** tuning walkthrough using a vector addition kernel, tied back to SMs, partitions, warps, and cores.

1. Baseline vector add kernel

We'll use the standard “hello world” CUDA kernel: each thread adds one element. [\[1\]](#) [\[2\]](#) [\[3\]](#)

```
__global__ void vecAdd(const float* a,
                      const float* b,
                      float* c,
                      int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        c[idx] = a[idx] + b[idx];
    }
}
```

Host code (simplified): [\[2\]](#) [\[3\]](#) [\[1\]](#)

```
int N = 1'000'000;
int threadsPerBlock = 256;
int numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;

vecAdd<<<numBlocks, threadsPerBlock>>>(d_a, d_b, d_c, N);
```

- Grid: numBlocks $\approx$ 3907.
- Block size: 256 threads $\rightarrow$ 8 warps per block (256 / 32). [\[4\]](#) [\[5\]](#)

This is our **starting point**.

This kind of grid/block diagram is often used to visualize how each thread maps to one element in vector addition. [\[2\]](#)

2. Step 1 – Choose an initial block size

We start with a reasonable default: 256 threads per block. ^{[6] [7]}

Why 256?

- Multiple of 32 (warp size).
- Large enough to give several warps per block (8 warps) to each SM.
- Usually plays well with typical SM resource limits. ^{[8] [9] [6]}

Hardware mapping:

- Every block is assigned to a single SM.
- Each block becomes 8 warps.
- These warps are distributed across the SM's partitions and scheduled on its cores. ^{[4] [5] [10]}

So with 3907 blocks and many SMs, the GPU has plenty of work: SMs pull blocks as they become free, and each SM holds many active warps.

3. Step 2 – Check occupancy for the baseline

Now we ask: how many warps are actually resident on each SM?

In a real workflow you would: ^{[8] [9] [11] [12]}

1. Compile the kernel with `-lineinfo` or debug symbols.
2. Open Nsight Compute, run the program, and inspect:
 - “Achieved Occupancy”.
 - “Block Limiters” (registers, shared memory, max warps, max blocks).
3. Or use the standalone CUDA occupancy calculator. ^{[13] [14] [11]}

Let's discuss conceptually what we might see:

- Vector add is simple; each thread uses few registers and no shared memory. ^{[1] [2] [3]}
- With 256 threads per block, many blocks should fit per SM.
- So occupancy is usually high, often near the hardware limit (e.g., many warps per SM). ^{[6] [12]}

Tied to the hierarchy:

- High occupancy means each SM has lots of warps resident.
- Each SM partition's warp scheduler has a healthy queue of warps to choose from.
- When one warp stalls on memory, the scheduler can quickly switch to another warp and keep the cores busy. ^{[4] [12] [15]}

If you found occupancy was surprisingly low, you would look at:

- Register count per thread (from compiler reports).

- Block size and whether a different value might allow more blocks per SM.
- Any shared memory usage (not present here).^{[9] [12] [8]}

4. Step 3 – Experiment with different block sizes

In practice, you'd benchmark with different block sizes:^{[13] [14] [6]}

- 64 threads per block.
- 128 threads per block.
- 256 threads per block.
- 512 threads per block.

You would measure:

- Kernel runtime (e.g., with CUDA events).
- Achieved occupancy and bandwidth (from Nsight Compute).^{[9] [11] [12]}

4.1 Expected patterns

- **64 threads/block (2 warps):**
 - Many blocks can be resident, but each block contributes few warps.
 - May hit the **max blocks/SM** limit before reaching max warps → medium occupancy.^[6]
^[12]
- **128 threads/block (4 warps):**
 - More warps per block; often good occupancy, good flexibility.^{[12] [6]}
- **256 threads/block (8 warps):**
 - Frequently near optimal; many warps, but still enough blocks to cover SMs.^{[7] [6]}
- **512 threads/block (16 warps):**
 - Fewer blocks can fit, but each block brings many warps.
 - Sometimes faster for simple kernels, sometimes not—needs measurement.^{[13] [16]}

You then **pick the fastest configuration**, not necessarily the one with the highest occupancy.^[9]
^[7]

Tied to SM partitions:

- Bigger blocks → more warps per block → more warps per SM and per partition.
- But if the block is too big, you may only fit 1–2 blocks per SM, which lowers flexibility when those blocks stall.^{[13] [6]}

5. Step 4 – Look at warp divergence

In this vector add kernel, divergence is minimal:

```
if (idx < N) {  
    c[idx] = a[idx] + b[idx];  
}
```

- Only threads in the last partially full block have `idx >= N`.^[1] ^[2] ^[3]
- That means almost all warps execute the same path: they all do the addition.
- For warps in the last block, some threads may skip the addition, but this is a tiny fraction of total work.

Tools like Nsight Compute can report **warp execution efficiency** (percentage of active threads per issued warp instruction).^[17] ^[18]

- For this kernel, you'd expect warp execution efficiency close to 100%.^[2] ^[17]

Tied to the hierarchy:

- Each warp is executed on an SM partition's cores.
- With almost no divergence, all lanes of each warp are active, so the cores are well utilized.
- Divergence would reduce how many of those 32 lanes do useful work on each instruction.^[19] ^[20]

So, for vector add, **divergence is not a performance concern**.

6. Step 5 – Memory access pattern and bandwidth

Vector add is memory-bound: the main cost is reading and writing arrays from global memory.^[2] ^[3]

The kernel accesses memory like this:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;  
c[idx] = a[idx] + b[idx];
```

Within a warp:

- Threads have consecutive `idx` values (e.g., 0–31, 32–63, ...).
- They load `a[idx]` and `b[idx]` for consecutive indices.^[3] ^[2]
- This gives **coalesced global memory accesses**, which is optimal.^[12] ^[21]

Profiling:

- Nsight Compute or Nsight Systems can show achieved memory bandwidth.
- You can compare that to theoretical peak bandwidth of the GPU.^[13] ^[17]
- For a well-written vector add, you should see high bandwidth utilization because:

- Accesses are coalesced.
- Occupancy is high.
- Divergence is minimal.^[21] ^[2] ^[12]

Tied to hardware:

- Each warp's memory instruction becomes a few memory transactions.
- The memory system feeds data to SMs; SM partitions then schedule warps on cores.
- High bandwidth + high occupancy ensures cores do useful work almost all the time.^[18] ^[12]
^[21]

7. Tuning loop: how you'd iterate in practice

Here is a concrete workflow you could follow for this or any simple kernel:

1. Baseline implementation (what we have now):^[1] ^[2] ^[3]

- One thread per element.
- Block size 256 (or 128).

2. Profile the baseline:^[17] ^[9] ^[11]

- Check:
 - Kernel runtime.
 - Achieved occupancy.
 - Warp execution efficiency.
 - Memory throughput.

3. Adjust block size:

- Try 128, 256, 512 threads per block.
- Re-profile each one.
- Choose the best runtime, but also note occupancy and bandwidth trends.^[13] ^[6] ^[7]

4. Check for divergence:

- For vector add, nothing to fix; for more complex kernels, use warp execution efficiency metrics to see if branches are hurting you.^[19] ^[20] ^[17]

5. Check for memory issues:

- Confirm accesses are coalesced (Nsight metrics, bandwidth).
- If not coalesced, adjust indexing or data layout.^[12] ^[21]

6. Stop when changes no longer help:

- As soon as further block size tweaks or occupancy tweaks don't improve runtime, move on.
- Remember: max occupancy is not automatically best performance.^[7] ^[9]

8. How this example maps back to SM/partitions/cores

Using the tuned vector add with, say, 256 threads per block:

- **Programming side:**
 - Grid: many blocks.
 - Block: 256 threads → 8 warps.
- **Hardware side:**
 - SMs: each holds multiple blocks, giving many resident warps.
 - SM partitions: each gets a subset of those warps and uses a warp scheduler to pick ready warps. ^{[4] [22]}
 - Cores: execute add instructions for all active threads in the warp in lockstep. ^{[10] [23]}

Because:

- Block size is reasonable.
- Occupancy is high enough.
- Divergence is negligible.
- Memory is coalesced.

...each SM partition spends most of its time issuing useful warp instructions to its cores, and the GPU achieves near-peak memory throughput for this operation. ^{[2] [18] [12] [21]}

If you want, we can next take a **slightly more complex kernel** (like a 1D stencil or 2D image blur) and repeat the same process, where divergence and shared memory usage start to matter more clearly.

✱

1. https://github.com/olcf-tutorials/vector_addition_cuda
2. <https://www.learnpdc.org/PDCBeginners/4-cuda/4-VectorAdd.html>
3. <https://eunomia.dev/others/cuda-tutorial/01-vector-addition/>
4. <https://doc.sling.si/en/workshops/programming-gpu-cuda/02-GPU/01-exemodel/>
5. <https://developer.codeplay.com/products/computecpp/ce/2.6.0/guides/sycl-for-cuda-developers/execution-model.html>
6. <https://moderngpu.github.io/performance.html>
7. <https://news.ycombinator.com/item?id=41808013>
8. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
9. <https://www.youtube.com/watch?v=OIOx3CJP2Es>
10. <https://www.abhik.ai/concepts/gpu-computing/shared-multiprocessor>
11. <https://christianjmills.com/posts/cuda-mode-notes/lecture-008/>
12. <https://www.olcf.ornl.gov/wp-content/uploads/2020/04/04-CUDA-Fundamental-Optimization-Part-2.pdf>

13. <https://forums.developer.nvidia.com/t/occupancy-optimazation-how-to-use-occupancy-calculator-improve-performance/25056>
14. <https://stackoverflow.com/questions/11735035/understanding-the-occupancy-calculator>
15. https://ajdillhoff.github.io/notes/cuda_architecture/
16. https://www.reddit.com/r/CUDA/comments/1ekin72/which_cuda_block_configuration_is_better_for/
17. https://ajdillhoff.github.io/notes/profiling_cuda_applications/
18. <https://arxiv.org/html/2501.16909v1>
19. https://www.youtube.com/watch?v=ypz8hOZ_xLU
20. <https://www.sciencedirect.com/topics/computer-science/warp-divergence>
21. <https://www.youtube.com/watch?v=PRtg7KqVs4A>
22. <https://modal.com/gpu-glossary/device-hardware/warp-scheduler>
23. <https://docs.modular.com/glossary/gpu/streaming-multiprocessor/>
24. https://www.youtube.com/watch?v=uUEHuF5i_qI
25. <https://www.scribd.com/document/982907519/numericals-2>
26. <https://www.youtube.com/watch?v=2NgpYFdsduY>